Assignment :- 6
Parallel and Distributed Computing
Submitted by
Pranav Chaudhary (102303320)

Q1:-

Code:-

```
#include <stdio.h>

int main() {
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    if (deviceCount == 0) {
        printf("No CUDA compatible devices found.\n");
        return 0;
    }

    for (int dev = 0; dev < deviceCount; ++dev) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, dev);

        printf("Device %d: \"%s\"\n", dev, deviceProp.name);
        printf("Compute capability:          %d.%d\n", deviceProp.major, deviceProp.minor);
        printf("Maximum block dimensions:    (%d, %d, %d)\n", deviceProp.maxThreadsDim[0],
                                                              deviceProp.maxThreadsDim[1], deviceProp.maxThreadsDim[2]);
        printf("Maximum grid dimensions:     (%d, %d, %d)\n", deviceProp.maxGridSize[0],
                                                              deviceProp.maxGridSize[1], deviceProp.maxGridSize[2]);
        printf("Maximum threads per block:   %d\n", deviceProp.maxThreadsPerBlock);
        printf("Total global memory:         %.2f GB\n", (float)deviceProp.totalGlobalMem / (1024.0 * 1024.0 * 1024.0));
        printf("Shared memory per block:     %lu bytes\n", deviceProp.sharedMemPerBlock);
        printf("Total constant memory:       %lu bytes\n", deviceProp.totalConstMem);
        printf("Warp size:                   %d\n", deviceProp.warpSize);
        printf("Concurrent kernels:          %s\n", deviceProp.concurrentKernels ? "Yes" : "No");
    }
    return 0;
}
```

Output of Program:-

```
Device 0: "NVIDIA GeForce GTX 1650"
Compute capability:          7.5
Maximum block dimensions:    (1024, 1024, 64)
Maximum grid dimensions:     (2147483647, 65535, 65535)
Maximum threads per block:   1024
Total global memory:         3.62 GB
Shared memory per block:     49152 bytes
Total constant memory:       65536 bytes
Warp size:                   32
Concurrent kernels:          Yes
```

1. What is the architecture and compute capability of your GPU?
   Architecture: Turing

Compute Capability: 7.5

2. What are the maximum block dimensions for your GPU?
   Maximum block dimensions: (1024, 1024, 64)

3. Suppose you are launching a one dimensional grid and block. If the hardware's maximum grid dimension is 65535 and the maximum block dimension is 512, what is the maximum number threads can be launched on the GPU?
   65535×512=33,553,920 threads

4. Under what conditions might a programmer choose not want to launch the maximum number of threads?
   - If the dataset is smaller than the maximum thread count.
   - If each thread requires a large amount of local resources (registers or shared memory), launching maximum threads would cause register spilling or fail to launch entirely.
   - To optimize cache hits and memory bandwidth (too many threads can thrash the cache).

5. What can limit a program from launching the maximum number of threads on a GPU?
   Registers per block: If a kernel uses too many variables, it hits the register limit before the thread limit.
   Shared memory per block: If threads request too much shared memory, fewer blocks can be scheduled on a Streaming Multiprocessor (SM).

6. What is shared memory? How much shared memory is on your GPU? Shared Memory is an extremely fast, on-chip programmable cache that is shared strictly among threads within the same block. It is used for inter-thread communication and data reuse. Amount: 49152 bytes per block

7. What is global memory? How much global memory is on your GPU? Global memory is the main, largest memory space on the GPU. It is accessible by all threads across all blocks, but it has high latency (slow) compared to shared memory.
   Amount: 3.62 GB

8. What is constant memory? How much constant memory is on your GPU? Constant memory is a small, read-only memory space cached on-chip. It is optimized for situations where all threads in a warp read the exact same memory address simultaneously.
   Amount: 65536 bytes

9. What does warp size signify on a GPU? What is your GPU's warp size?
   It is the fundamental unit of execution on an NVIDIA GPU. The hardware executes instructions in groups of threads called warps. Threads within a warp execute in lockstep (SIMT: Single Instruction, Multiple Threads).
   Size: 32

10. Is double precision supported on your GPU?
    Yes, almost all modern GPUs (Compute Capability 1.3 and above) support double precision (64-bit floating point), though the ratio of double-precision to single-precision performance varies heavily by architecture.

Q2:-

Code:-

```cpp
#include <stdio.h>
#include <stdlib.h>

__global__ void arraySum(float *d_in, float *d_out, int size) {
    extern __shared__ float sdata[];

    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (i < size) ? d_in[i] : 0.0f;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(d_out, sdata[0]);
    }
}

int main() {
    int N = 100000;
    size_t bytes = N * sizeof(float);

    float *h_in = (float*)malloc(bytes);
    float h_out = 0.0f;
    for (int i = 0; i < N; i++) {
        h_in[i] = 1.0f;
    }

    float *d_in, *d_out;
    cudaMalloc(&d_in, bytes);
    cudaMalloc(&d_out, sizeof(float));

    cudaMemset(d_out, 0, sizeof(float));

    cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    int sharedMemSize = threadsPerBlock * sizeof(float);

    arraySum<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(d_in, d_out, N);

    cudaDeviceSynchronize();

    cudaMemcpy(&h_out, d_out, sizeof(float), cudaMemcpyDeviceToHost);

    printf("Sum is: %f\n", h_out);

    cudaFree(d_in);
    cudaFree(d_out);
    free(h_in);

    return 0;
```

jj

Output of program:-

```
Sum is: 100000.000000
```

Q3:-

Code:-

```cpp
#include <stdio.h>
#include <stdlib.h>

__global__ void matrixAdd(int *A, int *B, int *C, int width, int height) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (col < width && row < height) {
        int index = row * width + col;
        C[index] = A[index] + B[index];
    }
}

int main() {
    int width = 1024;
    int height = 1024;
    size_t bytes = width * height * sizeof(int);

    int *h_A = (int*)malloc(bytes);
    int *h_B = (int*)malloc(bytes);
    int *h_C = (int*)malloc(bytes);

    for (int i = 0; i < width * height; i++) {
        h_A[i] = 1;
        h_B[i] = 2;
    }

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);

    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((width + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (height + threadsPerBlock.y - 1) / threadsPerBlock.y);

    matrixAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, width, height);
    cudaDeviceSynchronize();

    cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);

    printf("Result at C[0]: %d\n", h_C[0]);

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);

    return 0;
}
```

Output of program:-

```
Result at C[0]: 3
```

1. How many floating operations are being performed in the matrix addition kernel?
   Since these are "large *integer* matrices", no floating-point operations (FLOPs) are performed. However, there are exactly N×M *integer* addition operations.

2. How many global memory reads are being performed by your kernel?
   There are two global memory reads per thread (A[index] and B[index]). Therefore, there are 2×(N×M) total global memory reads.

3. How many global memory writes are being performed by your kernel? There is one global memory write per thread (C[index]). Therefore, there are N×M total global memory writes.